OPERATIONAL RUNBOOK · 5 AUGUST 2026

The runbook to 100%

Eight stages from where the system is today to everything genuinely working. Each stage has the exact commands, what you should see, and a check that proves it before you move on.

TWO DIFFERENT FINISH LINES

Stages 1–4 make the lead pipeline complete: an enquiry is captured, validated, stored, announced and impossible to lose. About an hour of work plus a DNS wait. Do these now — every one of them is a prerequisite for everything after.
Stages 5–8 make the remaining screens real: integrations, phone, and an AI that answers actual calls. Those are gated on provider accounts and monthly spend, not on code.

Where you are right now

COMPONENT	STATE	WHAT THAT MEANS
Website + domain	LIVE	alsaitigrowth.com on Cloudflare, security headers active
contact-submit	OLD BUILD	Saves enquiries, but accepts invalid emails and tells nobody
lead-notify	404	Not deployed. No lead alert can be sent
health	404	Not deployed. Nothing can be monitored
events-consume	404	Not deployed. Nothing delivers events
Migrations 0006-0010	NOT APPLIED	Five migrations of work sitting unused

Everything in stages 1–4 is already written, tested and committed. None of it is running.

STAGE 1

Deploy what is already built

~15 minutes · no new accounts needed

Install the Supabase command line tool and connect it to your project. You only ever do this part once.

```
npm install -g supabase
supabase login
supabase link --project-ref jnxvwdcvnwigowafdxvl
```

Now apply the database changes, then the functions. **Database first.**

```
supabase db push
supabase functions deploy contact-submit lead-notify health events-consume
```

THE ORDER IS NOT A PREFERENCE

The new contact form writes columns that the migrations create. Deploy the functions first and **every submission fails** — which is worse than today, because right now they at least save. Database, then functions. Always.

Then prove it took. This needs no password and checks everything from outside:

```
node tests/verify-deploy.js
```

You should see

`passed: 25` and no failures.

If it still says *accepted an invalid email*, the function did not redeploy — run the deploy command again and watch for an error.

STAGE 2

Make the alerts actually send

~5 minutes work · then a DNS wait

You have added the Resend key. On its own it sends nothing — the function also needs a From address, a destination, and a secret for the event consumer.

```
supabase secrets set \
ALERT_FROM_EMAIL="Alsaiti Growth " \
LEAD_NOTIFICATION_TO="contact@alsaitigrowth.com" \
EVENTS_CONSUMER_SECRET="$(openssl rand -hex 24)"
```

Save that consumer secret somewhere — stage 3 needs it. If you do not have `openssl`, any long random string will do.

Verify your sending domain

Go to **resend.com/domains**, add **alsaitigrowth.com**, and choose the **eu-west-1** region so mail sits beside your database. It generates four records to add in Cloudflare:

TYPE	NAME	VALUE
MX	send	feedback-smtp.eu-west-1.amazonses.com (priority 10)

TYPE	NAME	VALUE
TXT	`send`	`v=spf1 include:amazonses.com ~all`
TXT	`resend._domainkey`	**`p=MIGfMA0GCSqGSIb3DQEB…`** `copy from your dashboard`
TXT	`_dmarc`	`already present — leave it alone`

THE DKIM VALUE IS NOT PRINTED HERE ON PURPOSE

It is a public key generated uniquely for your domain the moment you add it. Nobody can supply it without opening your dashboard — treat anyone who hands you a complete set of four as guessing. It is about 400 characters and **must not wrap onto a second line**.

If Cloudflare appends the domain automatically, enter `send`, not `send.alsaitigrowth.com`.

STAGE 3

Turn on the machinery

~15 minutes · in the Supabase and Cloudflare dashboards

a. Enable the scheduler

Supabase → Database → Extensions → enable **pg_cron** and **pg_net**. Then re-run `supabase db push` so the retention job schedules itself.

b. Schedule the event consumer

This is what delivers lead alerts. Run it in the SQL editor, with your secret from stage 2 in place of the placeholder:

```
select cron.schedule('alsaiti_events','* * * * *', $$
select net.http_post(
url := 'https://jnxvwdcvnwigowafdxvl.supabase.co/functions/v1/events-consume',
headers := jsonb_build_object(
'Content-Type','application/json',
'x-consumer-secret','PASTE_YOUR_SECRET_HERE'),
body := '{}'::jsonb)
$$);
```

Then confirm both jobs exist:

```
select jobname, schedule from cron.job;
```

You should see
Two rows: `alsaiti_events` (every minute) and `alsaiti_retention_sweep` (daily).
No rows means nothing delivers your alerts and nothing enforces the retention your privacy policy promises.

c. Watch it

Point any uptime service — Better Stack, Cronitor, UptimeRobot — at the health endpoint on a 5-minute interval, alerting on any non-2xx response. It returns 503 when degraded, so no other configuration is needed.

```
https://jnxvwdcvnwigowafdxvl.supabase.co/functions/v1/health
```

Without this, a stalled queue is invisible: leads keep saving, the dashboard keeps working, and nobody is told about any of them.

STAGE 4

Prove it, with evidence

~20 minutes · needs one extra account

a. A real enquiry, end to end

Fill in the contact form on your own website with a real address. Then check three things in order: the confirmation shows a reference like `AG-1A2B3C4D`; a row appears in `contact_submissions` with `notification_status = sent`; and the alert email arrives within about a minute.

b. Prove one customer cannot see another

Create a second account in Supabase → Authentication → Users, then run the test. It plants a lead in one workspace and tries every route the other account might use to reach it — list, read by id, filter by workspace, update, insert, delete — across five tables, then cleans up after itself.

```
SUPABASE_URL=https://jnxvwdcvnwigowafdxvl.supabase.co \
SUPABASE_ANON_KEY=sb_publishable_... \
A_EMAIL=owner-a@example.com A_PASS=... \
B_EMAIL=owner-b@example.com B_PASS=... \
node tests/isolation-live.js
```

You should see

`SEC-03` PASSES and the probe lead removed.
Any failure here is a stop-everything event — it would mean one customer can read another's enquiries.

AT THIS POINT THE LEAD PIPELINE IS FINISHED

Enquiries are captured, validated, deduplicated, stored per customer, announced durably and retained to policy. Nothing is simulated and nothing can be silently lost. This is the part that makes or loses money, and it is done.
Three screens will still honestly say *Demo* or *Needs setup* — integrations, phone and the AI receptionist. Stages 5 to 7 fix those.

STAGE 5

Make Integrations real

~30 minutes · free · unblocks the Integrations screen

The HubSpot connector is written end to end — sign-in, encrypted tokens, field loading, lead sync, retry history. It has never authorised because no credentials exist.

At developers.hubspot.com

- Create a developer account and an app named **Alsaiti Voice**.
- Register this redirect URL **exactly**, character for character:

```
https://jnxvwdcvnwigowafdxvl.supabase.co/functions/v1/crm-callback
```

Request these scopes and no more. HubSpot rejects the whole install if one is not enabled on the app:

```
crm.objects.contacts.read crm.objects.contacts.write
crm.objects.deals.read crm.objects.deals.write
crm.schemas.contacts.read crm.schemas.deals.read oauth
```

```
supabase secrets set HUBSPOT_CLIENT_ID=... HUBSPOT_CLIENT_SECRET=...
```

Then in the app: Integrations → HubSpot → Connect, approve, and send a test lead. The chip only turns **Connected** when a real provider call succeeds — the app will not let it say so otherwise.

STAGE 6

Make the Phone screen real

~1 hour + verification · costs money

All four Telnyx functions are written and deployed, with signature verification and duplicate-event protection. They have never run because no account is funded.

- Create and verify a Telnyx account, then add funds. Number rental and test calls both cost.
- Create a least-privilege API key — voice and number management only.
- Buy one number in your target area.
- Set the webhook URL on the number:

```
https://jnxvwdcvnwigowafdxvl.supabase.co/functions/v1/telnyx-webhook
```

```
supabase secrets set TELNYX_API_KEY=... TELNYX_PUBLIC_KEY=...
```

`TELNYX_PUBLIC_KEY` is the Ed25519 key from the portal. Webhook verification fails without it, and unverified webhooks are refused — so the number will simply never answer.

> **THE DATABASE WILL REFUSE A FALSE CLAIM**
>
> A phone route cannot be marked active without a stored, verified inbound test call. This is not a rule in the interface that a future change might forget — the database rejects the write. Make a real call to the number and let it record the result.

STAGE 7

Make the AI answer real calls

weeks, not hours · ongoing monthly cost

This is the largest remaining piece and the only one that needs a server running permanently. A browser demo and a serverless function are not a 24/7 telephone agent: real calls need call routing plus a process that stays connected.

The worker is built, in `voice-worker/`. Everything a business would be held to — understanding what the caller said, deciding urgency, when to transfer, and the guarantee that one call creates exactly one lead — is covered by 40 automated tests. One file, the connection to the voice platform, has never run against a real call and says so at the top.

What you need to provide

ITEM	WHY
A LiveKit account and its three keys	The realtime voice layer
Somewhere always-on to run the worker	Fly.io, Railway, or any container host
An OpenAI key	Speech recognition, conversation and the voice
A phone number pointed at the voice route	From stage 6
The number a human answers	Where urgent calls transfer to
A decision on call recording	Changes what must be stored and announced

```
cd voice-worker
npm install
npm test # 40 checks, no credentials needed
docker build -t alsaiti-voice-worker .
docker run --env-file .env -p 8080:8080 alsaiti-voice-worker
```

CONFIGURE THE FALLBACK BEFORE THE FIRST REAL CALL

Decide what the carrier does when the worker is unavailable — voicemail, or forward to a mobile. Set it up **before** going live, not after the first outage. A caller who hears nothing is worse than one who hears a voicemail greeting.

Nothing may be labelled **Live** until a real call from an external phone reaches the assistant, creates exactly one lead, and the transfer and fallback both behave. Ten call tests are listed in `voice-worker/README.md`.

STAGE 8

The things that are not code

a few days · mostly other people

TASK	WHY IT MATTERS
Have a solicitor read the privacy policy and terms	They were written to match what the system actually does. That is accuracy, not legal advice, and they now carry retention commitments and a liability cap.

TASK	WHY IT MATTERS
Native-speaker review of Spanish and Arabic	Those languages now include legal text. A machine-translated liability clause is a real risk.
Open the site on a physical iPhone	Safari’s audio rules are the one thing no browser test or emulator reproduces, and the voice feature depends on them.
Rotate the demo password	It sits in a public file, so rotating does not hide it. What matters is that the demo account cannot reach real customer data.
Delete the junk test row	`delete from contact_submissions where email = 'notanemail';`

What “100%” means, precisely

Tick these off. When every line is true, the claim is honest.

#	TRUE WHEN	STAGE
1	An enquiry submitted on the live site is saved, and you can quote its reference	1
2	That enquiry produces an email you actually receive	2
3	An invalid email address is rejected instead of stored	1
4	Submitting twice creates one row, not two	1
5	A lead alert still arrives if the browser tab is closed immediately	1 + 3
6	Two customer accounts genuinely cannot see each other’s data	4
7	Old data is deleted on the schedule the privacy policy promises	3
8	Something alerts a human when the system degrades	3
9	HubSpot shows Connected only after a real record synced	5
10	A phone number shows Active only after a real inbound test call	6
11	A real call to that number is answered and creates exactly one lead	7
12	A failed transfer still leaves the lead and raises a callback	7
13	No screen claims a state that has not been proven	all

Time and cost, honestly

STAGES	WHAT YOU GET	TIME	COST
1–4	A lead pipeline that cannot lose an enquiry	**~1 hour** + DNS wait	Free
5	Integrations genuinely working	~30 min	Free
6	A real phone number connected	~1 hour	Number rental + call charges
7	An AI that answers actual calls	**Weeks**	Hosting + LiveKit + per-minute AI
8	Legally and linguistically sound	Days	Solicitor’s fee

IF YOU ONLY DO ONE THING TODAY

Stage 1. It is fifteen minutes and it is the difference between enquiries landing in a drawer nobody opens and enquiries reaching you. Everything else in this document depends on it, and none of the work already done protects a single customer until it runs.

Never send an API key, password, access token or recovery code through chat, email or a screenshot — including to a developer. Enter them directly into the provider dashboard or with `supabase secrets set`.